Session 2: Staying in Control

Context, Parallel Sessions, Plan Mode, and Verification

Brady Allardice

UPF & IBEI

Where We Are

Session 1: What Claude Code is and how it works: permissions, models, context, and CLAUDE.md.

Session 2: Staying in control on a real research task.

Today you will:

1. Work within the context window, and run conversations in parallel
2. Build the county panel in plan mode, reviewing the plan before it runs
3. Learn the Swiss franc data, write your expectations, merge, and check it yourself
4. Learn how tests work, then turn your expectations into tests

The methods are straightforward. The skill is the workflow.

Start-of-Session Ritual

The same one command as every session. Copy it from the top of **github.com/bradyallardice/claude-code-workshop**.

Mac (Terminal)

```
curl -fsSL https://raw.githubusercontent.com/
  bradyallardice/claude-code-workshop/student/
  tools/start_session.sh | bash -s -- 2
```

Windows (PowerShell)

```
irm https://raw.githubusercontent.com/
  bradyallardice/claude-code-workshop/student/
  tools/start_session.ps1 | iex
```

- **Work only in** `/my_workspace`**.** The course repo is a mirror that the command resets.
- Files you have **not** edited are updated; files you **have** edited are never touched, and the newer version appears beside them as `<name>.new`.
- Today's exercises are all in the `session_2` folder inside `my_workspace`. The slides show which folder to open for each one.

▸ Command fails?

Why Manage Context? Three Problems, One Goal

Everything in the conversation is **context**: your messages, the files Claude reads, its own replies and tool output. A long, cluttered conversation causes three separate problems.

1. The window fills

The context window is a fixed size. When the conversation reaches it, Claude Code **auto-compacts**: it replaces older messages with a summary.

Details can be lost. Next slide.

2. Token usage

Each reply is generated from the **whole conversation so far**, so a long conversation costs more with every turn.

Tokens are what your 5-hour and weekly limits count (Session 1).

3. Model performance

Output tends to get worse as context grows, **before the window is full**: early instructions get less weight, and failed attempts and irrelevant files crowd the useful material.

Errors compound from there.

The goal is performance

Managing context means keeping the right information in view and little else. That gets the model's best work, and it also saves tokens and avoids lossy compaction.

What Happens When the Context Window Fills Up

As a conversation grows, Claude Code eventually runs out of room.
When this happens, it **auto-compacts**: it summarizes older parts of the conversation to free up space.

What auto-compacting does:

- Replaces earlier messages with a compressed summary
- Keeps recent messages and tool results intact
- Happens automatically. You will see a notice in the chat

The risk: Details from early in the conversation (variable definitions, decisions you discussed, constraints you mentioned) may be lost or simplified in the summary.

You can also compact manually:
Type `/compact` to force a summary and free up context space.

This is why CLAUDE.md matters: it survives compacting because it is reloaded from disk, not from conversation history.

Strategies for Working Within the Context Window

Recall from Session 1: Claude Code can only "see" a limited amount of text at once. Errors compound when it loses track of earlier decisions.

Five strategies, each aimed at all three problems: a full window, token usage, and model performance:

1. Keep tasks focused
2. Maintain project documentation
3. Summarize work between conversations
4. Validate iteratively
5. Write token-aware prompts

Strategy 1: Keep Tasks Focused

One topic, one conversation.

Avoid:

- "Clean the data, build the panel, run the regressions, and make tables"
- Long conversations that drift across topics
- Asking Claude Code to hold your entire project in its head

Instead:

- "Clean and reshape the raw census file"
- Start a new conversation for each distinct task
- Let each conversation do one thing well

Why

A focused task fits comfortably in the context window. A sprawling task forces Claude Code to forget earlier steps, and that is where silent errors enter.

What Counts as One Task?

A task is a unit of work with **one deliverable and one check**.

One task:

- "Reshape the census file to county-year"
- "Merge the survey onto the panel"
- "Run the baseline regression"

Several tasks in disguise:

- "Clean the data and run the regressions"
- "Get the analysis working"
- "Fix the merge and rename variables"

Three tests for a single task:

- You can state it in one sentence
- It produces one output you can inspect: a cleaned file, a table, a figure
- You know in advance how you will check that it is right

The check is the boundary

If you would want to inspect something halfway through (does the merge look right?), that point separates two tasks.

One Conversation per Task

Start a new conversation when

- The task is finished and you have checked its output
- You switch to a different dataset, file, or question
- Claude contradicts an earlier decision or asks again for something you already said
- You see the auto-compact notice

What carries over

- **Does not:** anything said only in the chat, such as variable definitions, decisions, constraints
- **Does:** files on disk: scripts, `CLAUDE.md`, your notes

A new conversation is not starting over

The project is still on disk. Before you close a conversation, write down anything the next one needs.

Multiple Claude Code Tabs

You can run **several conversations at once**. Each one has **its own history and context**.

How to open another one

- **Open in New Tab**: `Cmd+Shift+Esc` (Mac), `Ctrl+Shift+Esc` (Windows)
- Also in the Command Palette: **Open in New Window**, **Open in Side Bar**
- Drag the panel to either sidebar or the editor area

A useful pattern

- Main task in the sidebar
- Side tasks in separate tabs, e.g. "explain this script" while the main one is busy

Give each tab a different job

Two tabs editing the same file can step on each other. Split the work by file, and keep the same conversation open in only one place.

Keeping Track of Your Conversations

Status dot on the Spark icon

- **Blue:** a permission request is waiting for you
- **Orange:** Claude finished while that tab was hidden

So you can leave a long task running in one tab and still notice when it needs you.

Finding old conversations

- **Session history** button at the top of the Claude Code panel
- Search by keyword or browse by time
- Clicking one resumes it, or jumps to its tab if it is already open

A permission request that sits unseen in a hidden tab stalls that task. Check the dot.

Demo: Two Tabs at Once

Live demo — follow along on your own machine

Open your `session_2` copy in VS Code. Open two Claude Code tabs, paste one prompt into each, and send both.

Tab A: one narrow task

```
Read data/PLN_CHF_EUR_FXdata.csv and describe each column
in one line. Do not modify any files.
```

Tab B: a different narrow task, at the same time

```
Read data/PLN_CHF_EUR_FXdata.csv and report the first and
last dates and the number of rows. Do not modify any files.
```

Two independent, read-only tasks: each tab has its own clean context, and neither can disturb the other.

Demo: The Collision Round

Watch this one — don't run it yourself

Same two tabs. This time both write to **the same file**.

Tab A

```
Write the column descriptions you just made to
output/data_notes.md.
```

Tab B

```
Write the dates and row count you just made to
output/data_notes.md.
```

Watch for: whose content is in the file at the end? Did either tab notice that the other had changed it?

Files are the only memory the tabs share. When two sessions write the same file without checking, the last writer wins.

Parallel Sessions: Benefits and Risks

Benefits

- **Throughput** on independent tasks
- **A clean context** for each task
- **A fresh session is an independent checker:** it has not seen the other's reasoning

Risks

- **No shared memory except files.** A tab already open loaded CLAUDE.md when it started, so it won't see later edits unless it re-reads the file
- **Collisions:** two sessions write the same output, and the last writer wins
- **Split attention:** you review several streams of output at once
- **Stalls:** a hidden tab can sit waiting on a permission prompt
- **Cost:** token use scales with the number of sessions

The same holds for several ChatGPT or claude.ai chats: each one knows only what you gave it.

Strategy 2: Maintain Project Documentation

Claude Code loads `CLAUDE.md` **automatically** every time it starts. It reads everything else only when a task needs it, or when you point it there.

Give each kind of information its own file:

- `CLAUDE.md`: rules, conventions, directory map, and pointers to the files below
- `README.md`: for human readers: the question, data sources, how to reproduce
- `CODEBOOK.md`: variable definitions, units, sources
- `STATUS.md`: where you stopped and what is next (see Strategy 3)
- `docs/decisions.md`: dated log of what you decided and why

The Payoff

Claude Code starts every conversation already knowing your rules, and loads the detail only when a task needs it. Keep `CLAUDE.md` short, and keep each fact in one file.

Live Demo: What Claude Code Looks Like in Practice

Claude Code on a real project of mine: AI Adoption Prediction, how well sociodemographics predict individual AI adoption across countries.

The project:

- Research folder: `src/` (scripts), `data/`, `docs/`, `outputs/`, a README
- Three models so far; a `CLAUDE.md` that says: log every decision, no long jobs without asking

What to watch for:

- Where Claude Code sits in the VS Code window
- How it **reads** files in the project folder
- How it **proposes edits** as a diff
- How you **approve, reject, or redirect** each change

The prompt:

```
I want to know whether a random forest predicts AI use at work any better than
the models we already have. Add it as a fourth model in the US prediction
pipeline, tuned inside the inner CV loop the same way the boosting model is.

Before editing anything, tell me which files you need to change and why.
Then make the changes, and do a quick test run with 1 repeat that writes to a
scratch folder, so the real outputs in outputs/ and data/interim/ aren't
overwritten.
```

Just watch; you'll do this yourself in the exercise.

Starting a New Project in VS Code

VS Code has no "New Project" button. **A project is just a folder** you open.

1. **Create an empty folder** in Finder / File Explorer (or use **New Folder** inside the Open Folder dialog)
2. **Open it:** File → Open Folder
3. **Trust the folder** when VS Code asks. The Claude Code extension does not work in Restricted Mode
4. **Build the structure** in the Explorer panel: right-click → New Folder / New File
5. **Open Claude Code** in that window

A typical research layout:

```
my_project/
  README.md
  src/  scripts
  data/
  docs/
  outputs/
```

From a terminal, `code .` opens the current folder in VS Code.

One project = one folder = one VS Code window. Claude Code works on the folder you have open.

Strategy 3: Summarize Work Between Conversations

When you finish a conversation, the context is gone. The next conversation starts fresh.

Before ending a conversation:

1. Ask Claude Code to summarize what it did and what remains
2. Save that summary in a file such as `STATUS.md`, overwriting the old one
3. Start the *next* conversation with "Read STATUS.md first"

Example prompt:

```
Summarize what we did in this session: what files
were changed, what decisions were made, and what
still needs to be done.
```

Think of it like a handoff memo between research assistants.
The next "assistant" knows nothing unless you brief it.

Demo: Summarizing a Session

Live demo — just watch

I ask Claude Code for a handoff summary at the end of a real working conversation.

What I will do:

1. Ask for a summary of what was done and what remains
2. Save it to a file
3. Use it to start the next conversation

Strategy 4: Validate Iteratively

The Compounding Error Problem

If step 1 is wrong and you do not catch it, steps 2–5 build on that mistake. By the time you check the final output, the error is buried.

Check each step before moving to the next:

1. Clean the data → *verify row counts and distributions*
2. Construct variables → *spot-check values against known cases*
3. Run the merge → *confirm pre- and post-merge row counts*

Slower per step, but **much faster overall** because you never have to unwind a chain of errors you didn't catch.

We practice this in the validation section later today.

Strategy 5: Token-Aware Prompting

Every word in your prompt costs tokens. Make them count.

A good prompt tells Claude Code three things:

1. **What files matter:** "Read `data/raw/census.csv` and `scripts/clean.py`"
2. **What the goal is:** "Create a county-year panel with one row per county per decade"
3. **What constraints apply:** "Keep all 3,143 counties; do not drop rows with missing population"

Vague:

"Clean up the data and make it ready for analysis"

Specific:

"Read `census.csv`, drop rows where `year` < 1950, and save as `panel_clean.csv`"

The Rule

Be specific about *inputs*, *outputs*, and *constraints*. Be brief about everything else. Context is finite; spend it on what matters.

Demo: A Good Prompt

Live demo — just watch

The same task twice, each in a fresh conversation: a vague prompt, then one that names the files, the goal, and the constraints.

Vague

```
Clean up the exchange-rate data and get it ready for analysis.
```

Specific

```
Read data/PLN_CHF_EUR_FXdata.csv. Add a column post_peg: 1 if
the date is on or after 2015-01-15 (the day the Swiss National
Bank removed the peg), 0 before. Keep all 269 rows. Save the
result to output/demo_fx_flagged.csv and do not modify the
raw file.
```

Look for: which files? what goal, in one sentence? which constraints? And in the vague one: what did it decide that I never asked for?

The Core Workflow

	Step	Who	If something is wrong
1.	Describe	You	Rethink before sending
2.	Review plan	You *	Reject, re-instruct
3.	Generate & validate	Claude Code + You	Read the code, run the tests, re-instruct
4.	Save or re-instruct	You	Keep the result (git in Session 3)

* = checkpoint where you can catch a problem before anything gets saved.

Validation is not a separate step — it happens **continuously** throughout steps 1–3.

The pattern

When something goes wrong, go back to **whichever step actually fixes the problem**: a bad plan means rewriting the instruction (step 1), not just re-running the same one. Diagnose first, then go back to the right step.

Step 1: Describe — Write It Down First

Before you type anything into Claude Code, write down your instruction.
If you cannot state the task precisely on paper, Claude Code cannot execute it precisely in code.

A good instruction specifies four things (example: precinct election results)**:**

1. **Inputs:** "Read `data/precinct_results.csv`"
2. **Goal:** "Add up the votes to one row per municipality"
3. **Constraints:** "Do not drop precincts with missing turnout. Do not edit the raw file."
4. **Expected output:** "`output/municipal_results.csv`, 412 rows, one per municipality"

Vague:
"Clean up the precinct data and aggregate it"

Specific:
"Sum the votes in `precinct_results.csv` by `municipality_id`. Keep precincts with missing turnout; report how many. Expect 412 rows."

Be specific about inputs, outputs, and constraints. Be brief about everything else.

The Three Execution Modes

The most important configuration decision you will make.

Plan Mode: Reviews approach with you *before* acting

Default Mode: Acts, but asks permission for "significant" steps

Auto-Accept: Executes everything without asking

↓ More autonomy, less oversight ↓

How it works:

- Claude Code analyzes your request
- Proposes a detailed plan (files to read, changes to make, approach)
- Waits for your explicit approval before acting
- You review, approve, modify, or reject

When to use:
Any task where the *approach* matters, not just the output.
Data transformation, variable construction, merges, anything analytical.

Why It Matters

The plan is your checkpoint. This is where you catch "I am going to drop every row with a missing value" *before* it happens, not after 300 observations are gone.

How it works:

- Claude Code executes steps autonomously
- Pauses for actions *it* considers significant
- Uses its own judgment about when to ask

When to use:
Routine tasks in established projects where you trust the general approach.

The Risk for Academics

Claude Code's judgment about what is "significant" may not match yours.
It might pause before writing a file but **not** before silently recoding a variable.

How it works:

- Executes everything without asking
- Reads, writes, runs, and modifies freely
- Stops only on errors

When to use:

- File reformatting
- Boilerplate generation
- Documentation

When not to use:
Anything involving your data or analysis.
The speed gain is not worth the loss of checkpoints.

The Key Insight on Modes

Different Relationships with the Agent

Plan mode = you **direct** the work
Auto-accept = you **review** the work after the fact

For published research, directing is almost always safer than reviewing after the fact.

Why: Errors you *prevent* at the planning stage are cheaper than errors you *catch* in the output, or worse, errors you don't catch at all.

Live Demo: Plan Mode vs. Auto-Accept

Same prompt, two modes (each in a fresh conversation):

```
Compare the average CHF/PLN rate before and after the
peg was removed in data/PLN_CHF_EUR_FXdata.csv.
```

Run it:

1. **Plan mode:** read the plan, redirect it, approve
2. **Auto-accept:** the same prompt, no approval step

Watch for:

- What does a plan look like, and what decisions does it list?
- How do you evaluate it, and what do you change?
- What does the experience feel like in each mode?

Step 2: Review the Plan

In plan mode, Claude Code proposes an approach before acting.

What to look for:

- Does it read the right files?
- Does the merge/analysis strategy match what you specified?
- Does it handle edge cases (missing data, extra rows, reference categories)?
- Does it make any *analytical decisions* you did not ask for?

The key question

Is there anything in this plan that I did not explicitly ask for?
If yes: reject or modify before it runs.

You do not always need plan mode. For simple tasks — summary statistics, reformatting, boilerplate — steps 1–3 collapse into one. But when uncertain, *use it*.

Step 3: Generate and Validate

Claude Code writes and runs the code. Your job is not done.

Read what Claude Code produced:

- Does the script do what the plan said it would?
- Are variable names and file paths correct?
- Did it make any choices the plan did not mention?

Common surprises at this stage:

- Silently dropping rows with missing values
- Choosing a default reference category you did not specify
- Using unweighted estimation when you asked for weighted
- Adding "helpful" data cleaning steps you did not request

The habit

Read the code, not just the output. Output can look correct even when the code is wrong.

Step 4: Save or Re-instruct

If the output passes verification: keep it. Claude Code saves its changes to your files after each step, so the results are already on disk.

If something is wrong: identify the specific error and re-instruct. Do not ask Claude Code to "fix it" — tell it *what* to fix and *how*.

In Session 3 we will learn **git** — a tool that lets you snapshot your project before each change and revert if anything goes wrong. For now, know that the save step exists and that there is a better way to manage it coming soon.

The full cycle

Describe → Review plan → Generate & validate → Save or re-instruct

Repeat for each task in your analysis. One cycle per task, not one cycle for the whole project.

Exercise, Step 0 on a Mac: Open the `county_panel` Folder

1. Click **File** → **New Window**. **Do everything below in the new, empty window.**
2. Click **File** → **Open Folder...** A Finder-style window opens.
3. At the top middle of that window is a **folder name with a small arrow** (a drop-down). **Click it.**
4. In the list that drops down, click **your name** (the entry with the **house icon**). You are now in your home folder.
   No house icon in the list? Click **Documents** in the left sidebar, then click the drop-down again: your name is now in it.
5. **Double-click** `my_workspace`
6. **Double-click** `session_2`
7. Click `county_panel` **once**, so it turns blue. Do not double-click it.
8. Click **Open** (bottom right of the window)
9. If VS Code asks *"Do you trust the authors of the files in this folder?"*, click **Yes, I trust the authors**
10. Click **Claude Code** at the bottom right of the VS Code window to open Claude Code

Exercise, Step 0 on Windows: Open the `county_panel` Folder

1. Click **File** → **New Window**. **Do everything below in the new, empty window.**
2. Click **File** → **Open Folder...** A File Explorer window opens.
3. In the list on the **left side** of that window, click **This PC**
4. **Double-click** the drive whose name ends in **(C:)** (e.g. *Local Disk (C:)*)
5. **Double-click** `Users`, then **double-click** the folder with **your name**. You are now in your home folder.
6. **Double-click** `my_workspace`
7. **Double-click** `session_2`
8. Click `county_panel` **once**, so it is highlighted. Do not double-click it.
9. Click **Select Folder** (bottom right of the window)
10. If VS Code asks *"Do you trust the authors of the files in this folder?"*, click **Yes, I trust the authors**
11. Click **Claude Code** at the bottom right of the VS Code window to open Claude Code

Exercise, Step 0: Check You Opened the Right Folder

Look at the Explorer panel on the left of the new window (if it is hidden: **View** → **Explorer**).

- **Right:** the heading says `COUNTY_PANEL`, and right under it you see `data` and `CLAUDE.md`
- **Heading says `MY_WORKSPACE` or `SESSION_2`?** You opened a folder one or two levels too high. Click **File** → **Open Folder...** again and go all the way down to `county_panel`
- **Heading says `DATA`?** You opened a folder inside `county_panel`. Click **File** → **Open Folder...** again, go back up to `session_2`, and click `county_panel` once
- **No `county_panel` inside `session_2`, or no `my_workspace` at all?** Run today's start command again, then repeat Step 0

Why the exact folder matters

Claude Code reads the `CLAUDE.md` of the folder you open. Open a folder above `county_panel` and it starts without the project's rules.

Full path, if you need it. Mac: `/Users/<your name>/my_workspace/session_2/county_panel`. Windows: `C:\Users\<your name>\my_workspace\session_2\county_panel`

Exercise: Build the County Panel From Scratch

Pairs, plan mode, in the `county_panel` window.

Where things are (in `county_panel/data/`):

Data: `election/countypres_sample.csv`; `ipums/census_YYYY_sample.csv.gz` (four files; some hold several survey years, see `YEAR`)

Codebooks: `election/County Presidential Returns 2000-2024.md`; `ipums/codebooks/census_YYYY_raw.xml`

1. Read the `CLAUDE.md` that is already in the folder (2 min)
2. Switch Claude Code to **plan mode**
3. Ask it to build a county-year panel that merges election returns with IPUMS census demographics
4. **Before approving the plan**, decide and check:
   - What the codes mean: look them up in the codebooks, for every variable you use
   - How each variable is defined: which vote share? how does education become "college"?
   - The merge: which keys, which join type, and how many rows you expect
5. Review the plan, reject or revise as needed, then approve
6. Run the script and verify the output in `output/`

Evaluating the Plan

When Claude Code proposes its plan, ask yourself:

- Is it reading the **codebooks**, or guessing what the codes mean?
- How does it turn `EDUC`, `RACE`, `HISPAN` into shares? What does it do with "not reported" codes?

  Do those match *your* definitions?
- Which election rows count? Did you check what values `mode` takes in each year?
- Which survey years go into each election year? Did you look at `YEAR` in each file?
- Are there census rows with no identifiable county? How does the plan treat them?
- Which join? What happens to counties that are in only one dataset?

This is where your domain knowledge matters.

Claude Code can write the code, but you decide what the variables mean.

Debrief

- What variable coding decisions did you make? Did Claude Code's default match yours?
- Did anyone reject or modify the plan? What did you change?
- Which `CLAUDE.md` rules showed up in the plan? Did it stop and ask about the open decisions, or pick for you?

The Lesson

A CLAUDE.md sets the rules before the task starts. Check whether the plan follows them.

Reviewing plans forces you to think about approach *before* you see output.

Domain knowledge is key for preventing errors.

How to Validate Your Data Work

Code that runs is not code that is right. Validating means checking the output against what you knew *before* you saw it.

1. **Form expectations:** from the codebook, the raw files, what you know about the data
2. **Write them down,** before any code runs
3. **Run the code, and read it:** does it do what you asked, nothing more?
4. **Check the output against your list,** yourself: count rows, look for repeats
5. **Trace a few records** from the raw file to the output
6. **Investigate every mismatch:** a bug or a misunderstanding? Never edit the expectation to fit

Steps 1 and 2 are the part you never delegate.

Verification: Code Can Run and Still Be Wrong

Claude Code can write code that runs, prints a plausible-looking table, and is still **scientifically wrong**.

None of these raises an error:

- A merge that duplicates rows when keys repeat
- A regression that silently drops every row with a missing control
- Standard errors clustered at the wrong level
- A reference category chosen alphabetically instead of by design

Two different questions

Does the code run and do what you intended?
Are the numbers *right*? Only the second one tells you about your research.

Read the Code

Claude Code is not deterministic — the same prompt can produce different code each time. But **code itself is deterministic.** If you read the code and understand what it does, you can be confident in the results, the same way you would be with code you wrote yourself.

This is the most fundamental verification:

- Read the script Claude Code produced, line by line
- Confirm it does what you asked — not more, not less

The principle

You are not trusting Claude Code. You are trusting *the code*. Reading does not scale to every number, which is why you also check the output against your expectations.

Form Your Expectations First, and Write Them Down

The Pre-Registration Analogy

Just as you pre-register hypotheses before seeing results, write down what a correct transformation should produce *before* you run it.

Before a merge or transformation, state:

- Expected rows $\times$ columns
- Which columns, and their types
- How missing values are handled
- A row you can trace by hand

Why write them down:

- Once you have seen the output, almost any number looks plausible
- An expectation in your head drifts to match what you see. One on paper does not
- It is what you check the output against afterwards

A mismatch is either a bug or a misunderstanding. Both are worth catching early.

Your Expectations Template

Every check starts as a sentence you write yourself. You write yours in `expectations.md`, already in your `session_2` folder. Each expectation has the same four lines:

```
# Expectations

## The file I am checking

## Expectation 1
- What must be true:
- How I know:
- How to check it:
- What I found:

## Expectation 2
...
```

What must be true: one plain sentence

How I know: the codebook, the raw file, or something you counted by hand. Never Claude Code's output

How to check it: what you will look at

What I found: filled in *after* the code runs. Never edit the lines above it

The first three lines are written before the merge. "What I found" comes after you check the merge yourself.

How Many Expectations? When Am I Done?

There is no fixed number. You are done when every step the code takes has an expectation that would fail if that step went wrong.

1. **List what the code will do** to the data, step by step: keep some rows, merge two files, recode a variable, drop missing values...
2. **For each step, ask:** if this went wrong, what would I see? Write that down as an expectation.
3. **Add a person or two** you looked up by hand in the raw file.

Last check: go down your list of steps. A step with no expectation? Write one.

Example: the county panel you just built

The code keeps only some election rows, merges them with the census, and computes vote shares. Three steps, so at least three expectations: one for the filter, one for the merge, one for the shares. Plus a county you look up by hand.

For one merge, this usually comes to about 5–8.

Today's Data: The Swiss Franc Shock

Background: In January 2015, the Swiss National Bank removed its currency peg. The Swiss franc surged against the Polish zloty. Polish households with CHF-denominated mortgages saw their payments spike overnight.

The survey: 2,044 Polish adults, interviewed October 2015. Respondents were randomly assigned to receive different information about the crisis before being asked whether the government should intervene to help affected borrowers.

Your files:

- `swiss_franc_survey.csv` — 2,044 respondents, 11 variables
- `respondent_demographics.csv` — income and ideology data (more rows than the survey)
- `codebook.md` — variable descriptions

Source: Ahlquist, Copelovitch, and Walter, "The Political Consequences of External Economic Shocks: Evidence from Poland," *American Journal of Political Science* (2020).

The Research Question

Does providing information about the Swiss franc crisis increase support for government intervention to help affected borrowers?

Treatment (randomized):

- Control (no info)
- Factual information
- Historical context
- Hungary's policy response

Outcome:

- Supports government intervention (binary)

Moderator:

- FX loan exposure status

The methods are straightforward. The skill is staying in control of the workflow.

Exercise Part 1: Learn the Data, Merge, Check, Then Write Tests

Learning the data is the part you never delegate. Write down what a correct merge must look like *before* Claude Code touches the data, check its work against your list yourself, and only then turn your list into tests.

0–10 min	Step 1: learn the data and write your expectations, **without Claude Code**
10–18	Step 2: merge and explore in plan mode
18–24	Step 3a: check the merge yourself against your expectations
	Then I show you how to turn checks into tests (about 20 min)
+0–6	Step 3b: turn your expectations into tests, and test them
+6–10	Debrief
+10–14	Step 4: write your own verification rules in `CLAUDE.md`

In Session 4 you will see how to hand the *checking* to Claude. The learning stays with you.

Part 1: Open session_2 and Find Your Files

Part 1 runs in the session_2 folder, not in county_panel.

1. **Open session_2 in its own window.** Same clicks as Step 0 (Mac, Windows): File → New Window, File → Open Folder…, go to your home folder, double-click my_workspace, then click session_2 **once** and click **Open** (Mac) or **Select Folder** (Windows).
   Check: the Explorer panel on the left is headed SESSION_2.
2. **Find the files in the Explorer panel.** Click a folder to open it; click a file once to open it.

What	Click in the Explorer panel
The codebook	docs → codebook.md
The survey	data → swiss_franc_survey.csv
The demographics	data → respondent_demographics.csv
Your expectations template	expectations.md (not inside a folder)

Tip: to read the codebook with its formatting, right-click codebook.md → **Open Preview**.
Full path: my_workspace/session_2/docs/codebook.md

Part 1, Step 1: Before Claude Code Touches the Data

1. **Learn the data and write your expectations, without Claude Code.** Read `docs/codebook.md` and open both CSV files. Then fill in the template in `expectations.md` (the template on the previous slides). Cover:
   - How many rows should the merged file have, and which key links the two files?
   - Which join type, and why? What happens to survey respondents with no demographics row?
   - Which variables have missing values, and how many do you expect?
   - What does `income_quintile` $= 0$ mean, and how must it be treated?
   - For each answer: how would you check it?

Write them now, before the merge. Afterwards, add what you found under each one, but never edit the expectation itself.

Part 1, Steps 2–3a: Merge, Then Check It Yourself

2. **Merge** the demographics data into the survey, in plan mode.
   - **As a script:** ask Claude Code to write the merge in `scripts/`, in your language (`merge.do`, `merge.R` or `merge.py`), with the two input file paths and the output path set at the top, and to run it to make `output/merged_survey.csv`
   - You decide the join type. Do not let Claude Code choose.
   - Then explore: summary statistics, treatment group sizes, outcome distribution.

3a. **Check it yourself, without Claude Code.**

   - For each expectation, run the count or look up the row yourself (Stata `count`, R `nrow()`, or open the CSV). Fill in *What I found*: met or not met, and what you saw
   - Trace three respondents: typical, missing outcome, absent from the demographics file
   - An expectation was wrong? Say so under *What I found*. **Do not edit the expectation**

Stop here. Next I show you how to turn what you just checked into tests.

You have no `CLAUDE.md` rules yet. Notice where Claude Code cuts a corner: you will write your rules from that.

Automating Verification: From Hand Checks to Tests

You just checked your merge by hand. That works once. But the code will change: you fix a bug, add a variable, get a new wave of data. Every change means redoing every check.

A test is a hand check written down as code, so the computer reruns it in seconds, every time.

By hand (what you just did)	**As a test** (what comes next)
You count the rows	The script compares the row count with your number
You look up one respondent	The script checks that respondent's values
Done once, when you remember	Rerun after every change
You read the result	It prints PASS or FAIL for each expectation

Your `expectations.md` is most of a test suite already. Tests do not replace your expectations: they hold them.

What a Test Is in Code: Where It Lives and How It Runs

Two files, two jobs. One script *makes* the data. A separate script *checks* it, and never changes it.

```
my_project/
  data/
  scripts/
    merge.do
  tests/
    check_merge.do
  output/
    merged.csv
  expectations.md
```

`data/` holds the raw files, never edited. In R or Python: the same layout, with `.R` or `.py` files.

How it runs:

1. Run `merge.do`: it writes `output/merged.csv`
2. Run `check_merge.do`: it reads that file and prints one PASS or FAIL line per expectation
3. Change the merge? Run both again

Why a separate file:

- The checks survive a rewrite of the analysis code
- One run checks *everything*, not just what you remember
- You, a coauthor, and Claude Code can all see what is checked

What a Test Is in Code: Inside the File

One block per expectation: the check, then a PASS or FAIL line. Plain Stata or R: no packages.

Stata: `tests/check_merge.do`

```
import delimited "output/merged.csv", clear

* Expectation 1: one row per person
capture isid person_id
display cond(_rc == 0, "PASS", "FAIL") ///
    " 1: one row per person"

* Expectation 2: 1,500 rows
capture assert _N == 1500
display cond(_rc == 0, "PASS", "FAIL") ///
    " 2: 1,500 rows, found " _N
```

R: `tests/check_merge.R`

```
d <- read.csv("output/merged.csv")
check <- function(ok, label)
  cat(if (ok) "PASS" else "FAIL", label, "\n")

# Expectation 1: one row per person
check(!anyDuplicated(d$person_id),
      "1: one row per person")

# Expectation 2: 1,500 rows
check(nrow(d) == 1500,
      paste("2: 1,500 rows, found", nrow(d)))
```

`capture` (Stata) and `check()` (R) let the file keep going after a FAIL, so you see *every* result, not just the first. Each number comes from your `expectations.md`, never from the output. You do not write this by hand: Claude Code writes it from your expectations, and you read it.

A One-Off Check vs. a Test

	One-off check	Test
Where	Typed in the console	Saved in `tests/`
When it runs	Once	After every change, all of it
Expected value	In your head	Written in the file
Result	You look, and remember	PASS or FAIL, printed every time

To turn a one-off check into a test:

1. Write the expected value down
2. Put the check in the test file
3. Rerun the *file*, not the line

Tracing a record by hand is a one-off check. It becomes a test when you save that record's ID and its expected values in the test file.

Layer 1: Software Tests Check That the Code Works

Test	It asks	Example
Smoke	Does the project run at all?	Packages import; every input file loads
Unit	Does one function behave correctly?	The age-group recode puts 17 in "under 18" and 18 in "18–29"
Integration	Do the pieces fit together?	The cleaned data has every column the model script uses
End-to-end	Does the whole pipeline run?	Raw data to final table from a single command
Regression (software sense)	Did a change alter verified results?	After refactoring, N and coefficients are unchanged
Edge-case and negative	Do unusual or bad inputs fail loudly?	A duplicate ID stops the merge instead of multiplying rows

Layer 2: Research Tests Check That the Results Are Right

Test	It asks	Example
Data integrity	Are keys, types, and ranges valid?	The ID column has no repeats; no age is below 0 or above 120
Merge integrity	Did the join keep and match what it should?	The row count after the merge is the number you wrote down; unmatched IDs on each side are counted
Sample accounting	Where did every row go?	N for each model, with a reason for every drop
Variable construction	Does each variable match its definition?	An "employed" dummy is 1 exactly for the codes the codebook lists as employed
Estimation specification	Is the model the one you intended?	The reference category is the one you chose; weights are used if the design needs them
Independent numerical check	Does a separate calculation agree?	A weighted share recomputed straight from the raw file

Independent Tests

A test only helps if it can fail for a reason the code cannot also fail for.

Independent:
Expected values come from outside the code under test: computed by hand from the raw file, by a different method, or from a published result.

Not independent:
Running the pipeline and pasting its output in as the "expected" value. The test can only confirm that the code agrees with itself.

Example prompt:

```
Write and run tests for these values, which I computed
from the raw file:
- Respondent <ID>: treatment = <value>, fx_status = <value>
- The control group has exactly <N> respondents
- The weighted share supporting intervention, among
  non-missing responses, is approximately <share>
Do not take expected values from the analysis script.
```

From Expectations to Tests

You wrote your expectations down before the merge, and checked the output against them by hand. A test is the same check, written once so the computer reruns it every time the code changes.

The expectation (a sentence you wrote)	**The test** (what must be true in the data)
Every interview took place during fieldwork	earliest date $\geq$ first day; latest date $\leq$ last day
A share lies between 0 and 1	every value of the share is ≥ 0 and ≤ 1
Treatment arms sum to the total sample	the arm counts add up to N
A new control changes N only through missing values	N falls by exactly the number missing in that control

Each expectation becomes one check. If a check fails, either the code or the expectation is wrong: find out which.

Trace Samples End to End

Aggregate checks can pass while individual records are wrong.
Follow specific records through every stage of the pipeline.

Raw file → Merged → Recoded → Estimation sample → Table

- Choose 3–5 respondents *on purpose*: typical, missing value, no match in file 2
- Print each one's values at every stage, side by side
- Tie every number in the final table (N, coefficients) to the rows behind it
- **Then save it:** each record's ID and expected values go into the test file, so the trace reruns with every change

Example prompt:

```
Pick one typical respondent, one with a missing outcome, and
one absent from respondent_demographics.csv. Show each one's
row in the raw, merged, and regression data. Then add a check
for each of them to the test file, using the values I confirm.
```

The Cycle: Expectations First, Check, Then Codify

Repeat for every component: merge, variables, model, table.

1. **You write the expectations** before any code runs: files, keys, row counts, edge cases
2. **Claude Code writes and runs the code**
3. **You check the output against your list, yourself:** count the rows, look up a few people, mark each expectation met or not met
4. **Turn your checks into a check script**, from your expectations file, so they rerun every time the code changes
5. **Failures are investigated, not patched:** fix the code, not the expected value. Rerun all the checks before the next component

Why expectations first

What keeps a check honest is *when you wrote the expected value*: before you saw the output. Write it down first, and never edit it to match what came out.

Test the Tests

A test that cannot fail is not a test. Inject a known error and confirm the check catches it.

Fault injection (always on a copy, never the raw data):

- Duplicate one survey respondent's row in a copy of the demographics file: the row-count and key tests should fail
- Change one treatment label in a copy: the spot check should fail

Planted answer: simulate outcomes with a known treatment effect. The estimator should recover it within a stated tolerance.

Example prompt:

```
Copy respondent_demographics.csv into output/ with one survey
respondent's row duplicated. Point the merge script at the
copy and run it, then run the check script unchanged and
show me which checks fail.
```

Passing Is Not the Goal

Claude Code must never make a test pass by:

- Editing expected values to match the output
- Deleting or loosening assertions
- Suppressing warnings
- Changing the specification: join type, reference category, sample, weights

Not every check is pass/fail:

- **Hard failure** (duplicate keys): stop
- **Warning** (missingness jumps from 4% to 11%): flag it for you
- **Judgment** (is identification credible?): yours

A failing test is information

Claude Code should report it and ask. It should not make it disappear.

What Tests Cannot Do

- **Catch mistakes they were not written for.** A "shares lie between 0 and 1" test passes on a merge that duplicated half the rows
- **Tell you your expectation was right.** Expect the wrong thing, and the test passes on the wrong answer
- **Make research decisions.** Which join, which sample, how to code an awkward answer: a test holds the choice you made, it cannot tell you it was right
- **Prove the code is correct.** Passing means what you wrote down still holds, nothing more
- **Keep up on their own.** Change the data or the design, and some expectations must change too: *on purpose*, with a written reason, never to match the output
- **Replace reading the code and looking at the data.** Those find what you did not think to test

"Program testing can be used to show the presence of bugs, but never to show their absence." — Edsger Dijkstra

Live Demo: Writing Tests, Step 1 — My Expectations, Checked by Hand

The same template you just used, for a file from my AI Adoption Prediction project, checked by hand:

```
## The file I am checking
data/analysis/us_hps2603_workers.parquet
## Expectation 1
- What must be true: every person appears once
- How I know: each person has one ID, in the column SCRAMID
- How to check it: no SCRAMID appears twice
- What I found: no SCRAMID repeats. Met
## Expectation 2
- What must be true: only people who worked last week are in the file
- How I know: I counted 6,680 of them in the raw survey file (ANYWORK = 1)
- How to check it: the file has exactly 6,680 rows
- What I found: 6,680 rows. Met
## Expectation 3
- What must be true: person 260300RfPHR7 does not use AI at work (y1 = 0)
- How I know: in the raw file they answered "no" to all 11 AI questions
- How to check it: their y1 is 0
- What I found: y1 = 0. Met
```

Each check is one line I run myself: count the rows, look for repeated IDs, look up one person.

Live Demo: Writing Tests, Step 2 — Turn My Checks Into a Script

I do not want to redo these by hand every time the code changes. So I write them down once, as a script:

```
Turn expectations.md into a check script, tests/check_workers.py:
one check per expectation, using only the values written there.
It runs every check and prints PASS or FAIL for each. Show me
the script before you run it.
```

My project is in Python. In Stata or R, ask for the same thing as a do-file or an R script: no extra packages needed.

- **Read each check:** does it test what my sentence says, using *my* number?
- **Run it:** `Run the check script.`
- **Three PASS lines**, matching what I found by hand

The script holds what I already checked. It does not decide what is right.

Live Demo: Writing Tests, Step 3 — Test the Tests

All PASS proves nothing yet. Would the script notice a mistake? Break a copy on purpose:

```
Make a copy of the data file with its last row duplicated.
Run the same check script on the copy, without changing it.
Which checks fail?
```

Fail (good):

- Expectation 1: one ID now appears twice
- Expectation 2: 6,681 rows, not 6,680

Still pass:

- Expectation 3: person 260300RfPHR7 is unchanged

A check only catches the mistakes it was written for. That is why you write several, each from a different sentence.

When and How Do I Run the Tests?

When: changed anything? Run them.

- Right after the code first makes the file
- Every time the code or the data changes. Run **all** the checks, not just the newest
- Before you use the output: a table, a figure, an email to a coauthor
- When you test the tests, on a broken copy

How, option 1: ask Claude Code

```
Run the check script and show me the output.
```

How, option 2: yourself. It is a normal script: open it in VS Code and run it the way you run any script **Stata:** run the do-file. **R:** source the `.R` file. **Python:** run the `.py` file.

In the Stata or RStudio app, first set the working folder to `session_2`: Stata `cd ~/my_workspace/session_2`, R `setwd("~/my_workspace/session_2")`. Windows: `C:\Users\<your name>\my_workspace\session_2`. Or ask Claude Code to run it.

Reading the result: one line per expectation.

```
PASS Expectation 1: every person appears once
FAIL Expectation 2: 6681 rows, expected 6680
```

Any FAIL: find out *why*. Do not change the check to make it pass.

Claude Code's summary is a report. The output is the evidence: look at it yourself.

Making Claude Code Run and Respect Your Tests

Put it in `CLAUDE.md` **once,** so every conversation follows it. Ask Claude Code to add a **Tests** section that tells it to:

- run your check script after *any* change to code or data, and show you the full output
- stop and tell you if a check fails: which one, and why
- never edit the tests or the expected values to make them pass

A good report has:

- The full PASS/FAIL list, not "all tests pass"
- Every FAIL named, with what it found
- What was *not* checked

Red flags:

- An edit to a file in `tests/` you did not ask for: **reject it**
- "Tests pass" with no output shown
- An expected value that changed to match the output

- **Keep tests where you see every edit:** in plan or default mode, Claude Code asks before changing a file. Any change to `tests/` needs a reason you agree with
- `CLAUDE.md` is an instruction, not a guarantee: **still read the output yourself**

Rules in CLAUDE.md Come From Failures

`CLAUDE.md` holds standing instructions: Claude Code reads it at the start of every conversation. A rule is worth writing only when you know the failure it prevents.

A good rule has three parts:

1. The failure you saw
2. The rule, in your own words, specific enough to follow
3. How you would notice that it was broken

Failures to watch for in Part 1:

- Tests whose expected values were copied from the code's own output
- A design choice (the join type) made for you, without asking
- A failing test "fixed" by changing the test
- "All tests passed" with no word on what was not tested

Do not write rules yet. After Part 1 you write your own, from what actually happened to you.
`CLAUDE.md` is an instruction, not enforcement: read the tests too.

Part 1, Step 3b: Turn What You Wrote Into Tests

Back in your `session_2` window. The same steps as my demo, in your own words:

1. **Ask Claude Code for a check script** made from your `expectations.md`: saved in the `tests` folder, in your language (`.do`, `.R`, or `.py`), printing PASS or FAIL for each expectation
2. **Read it before it runs:** does each check test your sentence, with your number?
3. **Run it.** Does it agree with what you found by hand? If not, find out why. Do not change the check to make it pass
4. **Make running it a rule:** ask Claude Code to add a **Tests** section to `CLAUDE.md` that says
   - run the check script after any change to code or data, and show the full output
   - if a check fails, stop and say which one and why
   - never edit the tests or expected values to make them pass
5. **Save your traced respondents too:** ask for a check for each respondent you traced (the ID and the values you confirmed), so the trace reruns with everything else

Part 1, Step 3b (continued): Test Your Tests

Would your script notice a broken merge? Break a *copy* on purpose. One instruction at a time:

1. **Make a broken copy.** Ask Claude Code to copy the demographics file into `output/` and, in the copy, duplicate the row of one respondent who is **also in the survey**. The original in `data/` stays untouched
2. **Merge with the broken copy.** Point the demographics path at the top of your merge script at the copy, and run the merge
3. **Run your check script, unchanged,** and read the full output. The row-count and one-row-per-respondent checks should FAIL. All PASS? Your checks cannot see a duplicate: add one that can
4. **Put it back.** Point the merge script at the original in `data/` again, run the merge, run the checks: all PASS
5. **Clean up:** delete the broken copy

Part 1 Debrief: What Did Your Expectations Get Wrong?

- Which of your expectations turned out to be wrong? Was the error yours, or Claude Code's?
- What did you learn about the data that the codebook did not say outright?
- Did Claude Code's tests check what you wrote, or something easier?
- Where did Claude Code cut a corner you had not asked for? Which of your rules would have prevented it?

The point

You can only verify work against expectations you hold. Writing them down is how you learn the data, and that part is not delegated.

Part 1, Step 4: Write Your Own Rules

Open the `CLAUDE.md` you started in Step 3b (it has your Tests section). Add **3–5 rules of your own, in your own words**, each from something that happened in Part 1.

For each rule, write:

1. **Failure:** what happened, or nearly happened
2. **Rule:** one specific instruction to Claude Code
3. **Check:** how you would notice it was broken

The shape of one (not one to copy)

Failure: a test expected N to equal the length of the script's own output. *Rule:* expected values come from `expectations.md` or the raw files, never from the script. *Check:* a hard-coded number that matches the output to the last digit.

Rules take effect in a new conversation. Keep the file: Session 3 builds on it and compares it with a reference.

Next Session

Session 3: Verification Through to the Paper

- Estimate the models and account for every observation, then robustness checks and a LaTeX table
- Git as the undo button: commit → instruct → diff → revert
- LaTeX in VS Code: from results to a paper that Claude Code can edit and check

Before next session:

- Keep your `my_workspace/session_2` folder as it is. Session 3 continues the exercise in it.
- Make sure `setup_check.tex` from Session 1 compiles (setup guide, Step 7)
- Try the core workflow on something from your own research, and note where Claude Code made a decision you did not ask for

Back Matter

Block	Min
If the start command fails	2

If the Start Command Fails

Run this in the course repo, then run the start command again (or copy the folder by hand, first time only):

```
cd ~/claude-code-workshop
git fetch origin
git reset --hard origin/student   # match the published version
```

This discards edits made *inside the course repo*; your `my_workspace` is not touched. Do not use `git pull`.

Back to the ritual

Back matter